Working with Explicit Wait WebDriver - Lecture - 5

Agenda

- Window and Navigation Interface in WebDriver
- Create Folder Script in app.box.com
- Verify Title Text in Popup
- Using JUnit Assertions
- Random Folder Name for Testing
- Select Class in Selenium WebDriver
- Verifying And Waiting For Message Invisibility
- Selecting Same Folder Row Element
- Deleting the Folder Row and Logging Out
- Quiz
- Homework

Window and Navigation Interface

- **Window Interface** – Inner Interface of **WebDriver** for getting, setting size and position of **WebDriver window**
  - **Options interface** has a method **window()**, which returns the reference of **Window interface**
  - Most used methods of **Window Interface** are **maximize()** and **fullscreen()**
- ***Getting Option interface's reference by calling manage() method of WebDriver interface:***
  Options opt = driver.manage();
- ***Getting Window interface's reference by calling window() method of Options interface:***
  Window win = opt.window();
- ***Calling maximize() method of Window interface:***
  win.maximize();
  OR
  No need getting reference of Options and Window interfaces separately:
  driver.manage().window().maximize();

- **Change outer window dimension using org.openqa.selenium.Dimension** ***class in WebDriver API:***
  Dimension dim = new Dimension(1000, 1000);
  win.setSize(dim);

- **Navigation Interface** – **WebDriver API** provides Navigation interface (**defined in WebDriver**) used for different navigations on web page
  - **WebDriver interface** contains **navigate()** method, which returns instance of **Navigation interface**
  - Methods of **Navigation interface** provides moving back and forward also refreshing in browser's history

// calling navigate() method of WebDriver returns Navigation interface's reference

```
Navigation nav = driver.navigate();
nav.to("url");          //browser navigate to any url
nav.refresh();          //refresh current browser page
nav.back();             //go  browser's history back page
nav. forward();         //go  browser's history forward
```

OR

// no need to get instance of Navigation interface, instead calling its method directly

driver.navigate().to("url");

Create Folder script in app.box.com

```
WebElement element = wait.until(ExpectedConditions.presenceOfElementLocated(By.name("login")));
element.sendKeys("qacult.demo@gmail.com");

WebElement submitEmail = wait.until(ExpectedConditions.presenceOfElementLocated(By.id("login-submit")));
submitEmail.click();

WebElement enterPassword = wait.until(ExpectedConditions.presenceOfElementLocated(By.id("password-login")));
enterPassword.sendKeys("testing123");

WebElement login = wait.until(ExpectedConditions.presenceOfElementLocated(By.id("login-submit-password")));
login.click();

Thread.sleep(2000);
String newBtnXpath = "//button[contains(@id,'menubutton')]//span[text()='New']";
WebElement newBtn = wait.until(ExpectedConditions.presenceOfElementLocated(By.xpath(newBtnXpath)));
newBtn.click();

String folderLinkCss = "li[aria-label='Create a new Folder']";
WebElement folderLink = wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector(folderLinkCss)));
folderLink.click();
```

Login to the app

Click on "New" button

Click on Create Folder link

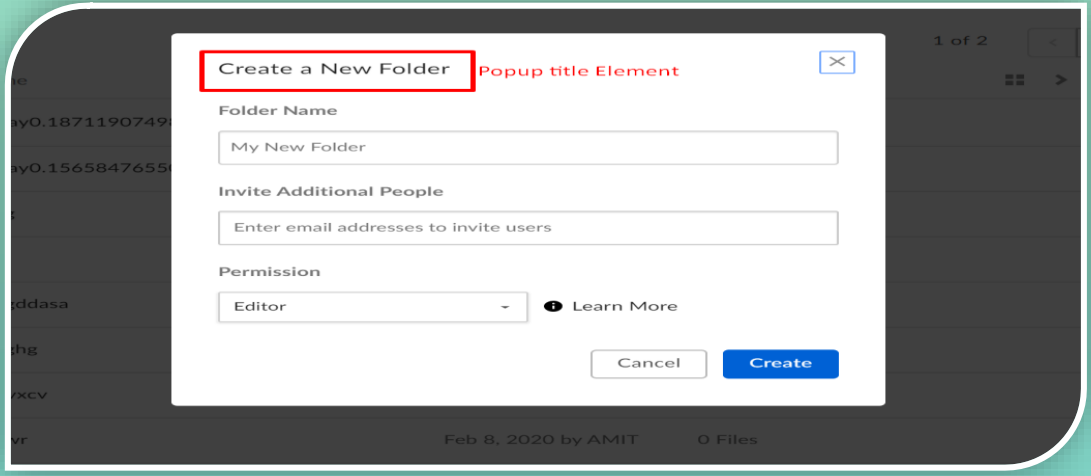
We are going to create **WebDriver** script for **creating and deleting** new folder in box portal. Steps to follow:

1. Navigate to http://app.box.com and **login** into app
2. Click on **New** and select **Create a folder**
3. In a **new popup,** get and **assert** popup text
4. **Input** Random folder name, **select** Permission as **viewer** and Click on “Create” button
5. Verify **success notification** and **wait** for it to **disappear** from screen
6. Now **select** newly created **folder row to click** on **Trash button** and then confirm to delete
7. Verify **deletion notification** and wait for it to **disappear** from screen
8. Finally **logout** from portal

Note: Please find all the elements dynamically using explicit wait

Verify Title Text in Popup

- **Popup** for new folder has a header title element with the text "**Create a New Folder**" which can be fetched with **getText()** method of **WebElement**
- User can verify the header title with **if-else condition**
- Instead of **if-else condition** we can use **Assertions** which will cause the program to end in case of a failure



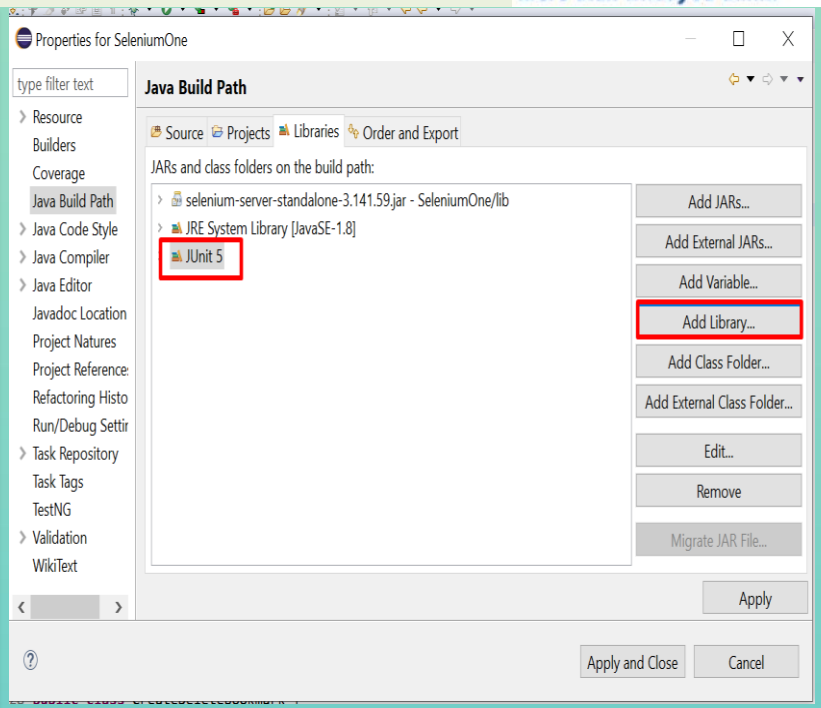
Verify with Assertion

```
boolean popTitleFlag = popupTitle.getText().equals("Create a New Folder");
Assert.assertTrue("Popup could not be opened", popTitleFlag);
```

```
WebElement popupTitle = wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div.modal-header")));

if(popupTitle.getText().equals("Create a New Folder"))
    System.out.println("Create Folder popup opened successfully");
else
    System.out.println("Popup could not be opened");
```

Using JUnit Assertions

- **Assertions** are **utility methods** to support conditions in tests.
- These methods **evaluate a conditional expression** at runtime.
- If the **conditional expression** is true, the **assert** statement does nothing.
- If the **conditional expression** evaluates to false, an error message is displayed and the program is terminated.
- These methods are accessible through ***org.junit.Assert*** class.
- An **AssertionError** is an **error** generated by the program to indicate that something that should never happen, has happened.
- **Assertion is an error** and not an exception and therefore can't be handled with **try-catch block.**
- To **import** Junit Assert class**, include Junit Library** in project's build path.



Some important assertion methods:

- **Assert.assertTrue(String message, boolean condition) –** Asserts that a given condition is true, If it isn't true it throws an AssertionError with the given message.
- **Assert.assertEquals(String message, Object expected, Object actual)** - Asserts that two objects are equal, If they are not it throws an AssertionError with the given message.
- **Assert.assertNotNull(String message, Object object)** - Asserts that given object isn't null, if null then it throws an AssertionError with the given message.

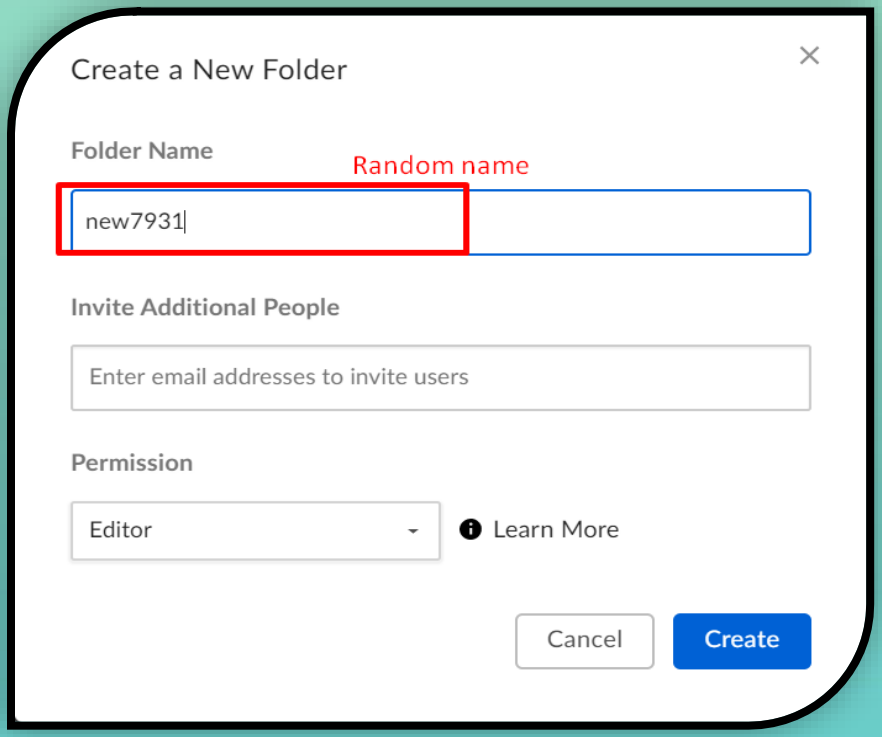
```
public static void main(String[] args) {
    boolean numberFlag = 5>7;
    Assert.assertTrue("assertion failed", numberFlag)

}
```

This will cause

```
Exception in thread "main" java.lang.AssertionError: assertion failed
        at org.junit.Assert.fail(Assert.java:88)
        at org.junit.Assert.assertTrue(Assert.java:41)
        at com.firstLogin.AssertTest.main(AssertTest.java:9)
Picked up _JAVA_OPTIONS: -Djava.net.preferIPv4Stack=true
```

Random Folder Name for Testing

- Creating folder with **same name** in the portal is not a good practice and also <u>not allowed</u> in **Box portal**.
- Generate **random number** with static method **random()** of **java.lang.Math** class.
  - **Math.random()** method returns **double** number between **0.0 and 1**.
- For our convenience, we converted the **double** value into **int** value with upto four digits.
- Append this random **int** value with a **String** value to convert it into **String**.
- This **String** value can be given as input to folder name **text-box**.




```
String nameToBeInput = "new"+ (int)(Math.random()*((1000-1)+1))+1;
WebElement folderName =wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("input[name=\"folder-name\"]")));
folderName.sendKeys(nameToBeInput);
```

Using Math.random to generate random value

Select Class in Selenium WebDriver

- **Select** is a class that is provided by **WebDriver API** to perform multiple operations on ***DropDown object*** and ***Multiple Select objects***.
- **Select class** models a ***SELECT*** tag in **HTML** and provides helper methods to select and deselect options.
- To use this class, it is required to import "**org.selenium.support.ui.Select**" package.
- Some important methods:
  - ***selectByVisibleText(String arg0) : void*** – To choose an option with given text under any dropdowns or multiple selection boxes
  - ***selectByIndex(int arg0) : void*** – To choose an option with index
  - ***getOptions( ) : List<WebElement>*** – To get all options within a dropdown as list of WebElement.
  - ***deselectAll( ) : void*** – To deselect all selected options in multiselect
  - ***deselectByVisibleText(String arg0) : void*** – To deselect with text of option in multiselect
  - ***isMultiple( ) : boolean*** – Tells whether the SELECT element supports multiple selecting options or not

Simple Select

```
▼<select name="invite-permission" data-resin-target="selectpermission">
  <option data-resin-option="Editor" value="Editor">Editor</option>
  <option data-resin-option="Viewer" value="Viewer">Viewer</option>
</select>
```

Multiple Select

```
<select class="js-example-basic-multiple" multiple="multiple">
  <option value="US">United States</option> == $0
  <option value="IND">India</option>
  <option value="GBR">Great Britain</option>
</select>
```

```
WebElement selection = wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("select[name=\"invite-permission\"]")));
Select permissionSelect = new Select(selection);
permissionSelect.selectByVisibleText("Viewer");
Thread.sleep(3000);
List <WebElement> elementCount = permissionSelect.getOptions();
System.out.println(elementCount.size());
for(WebElement sample: elementCount) {
    if(sample.getText().equals("Editor"))
        permissionSelect.selectByVisibleText("Editor");
}
```

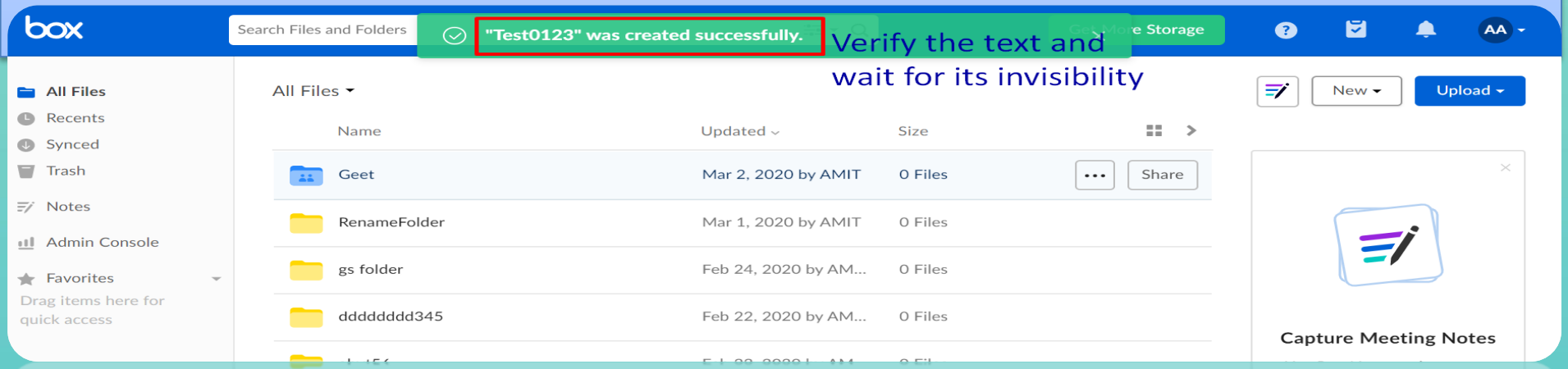
Create Select class from WebElement

Get all options in Select tag as WebElement

Select the option as visible Text

Verifying And Waiting For Message Invisibility

- Successful folder creation has a **notification message** which should be verified.
- We should find locator for the notification message in such a way that its locator:
  - Yields **zero results** when it is <u>not</u> present(visible) on the screen
  - Yield **one result** when it is visible on the screen
- We should get text of this **WebElement** and **assert**.
- Before performing any next action, we should wait for notification message **invisibility**.
- We can check for **WebElement's** invisibility using conditions:
  - **ExpectedCondition<Boolean> invisibilityOf(final WebElement element)**
  - **ExpectedCondition<Boolean> invisibilityOfElementLocated(final By locator)**

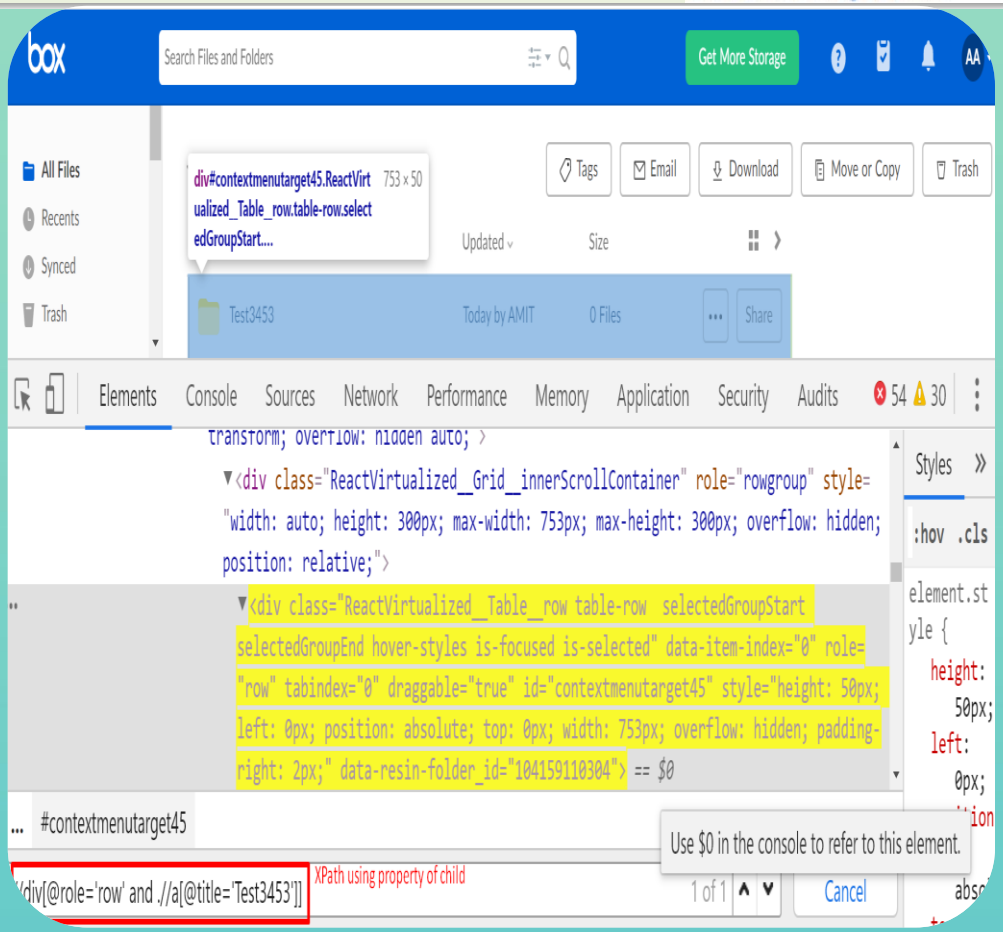



```
// To verify the success message and waiting for its invisibility
WebElement succcessMessageNotification = wait.until(
        ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div[class*='notification info wrap']")));
wait.until(ExpectedConditions.invisibilityOf(succcessMessageNotification));
```

Selecting same Folder Row Element

- Let's create **xpath locator** for the same **folder row**, we created earlier.
  - For this element, we have attribute "**role**" whose value is "**row**"
  - Using **role attribute** for this element returns **multiple** elements.
  - For making this locator unique, we have to use **child's property** .
  - We use **(.) operator** for using **child's property** but **staying** at parent level.
  - **"//"** is used to indicate that child may be **any level down**.

- In **WebDriver** script, name of folder which was used while creation can be concatenated in **Xpath** itself.




```
//select and delete folder row
wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//div[@role='row' and .//a[@title='"+nameToBeInput+"']]"))).click();
```

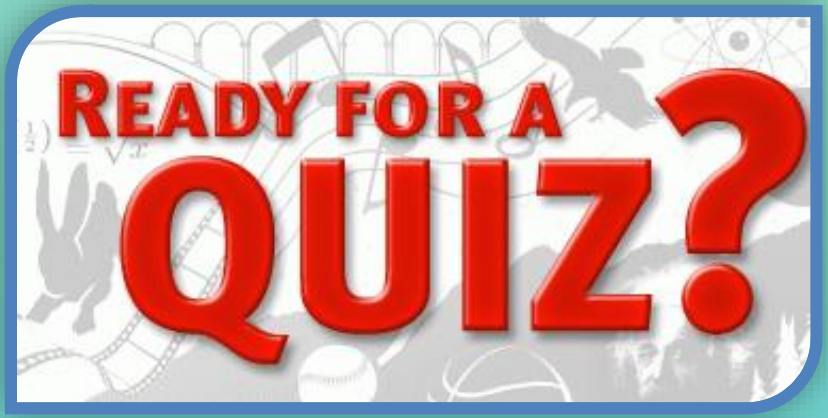
Deleting the Folder Row and Logging Out

- We have to **delete the same folder row** we created:
  - Select the folder row.
  - Click on **Trash**
  - **Confirm Deletion** in confirm dialog box.
- Verify the **deletion notification message** and get its text to assert.
- Wait for deletion message **notification invisibility**.
- After this, **logout** from the box portal.

```
//same folder row selected and deleted
wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//div[@role='row' and .//a[@title='"+nameToBeInput+"']]"))).click();

wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("button[data-resin-target=\"trash\"]"))).click();
Thread.sleep(1000);
wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//button[@data-resin-target=\"primarybutton\"]"))).click();

//verify deletion message
WebElement succcessMessageNotification1 = wait.until(
        ExpectedConditions.visibilityOfElementLocated(By.cssSelector("div[class*='notification info wrap']")));
System.out.println(succcessMessageNotification1.getText());

Assert.assertTrue("folder not deleted succcessfully", succcessMessageNotification1.getText().contains("Item successfully moved to trash."));
wait.until(ExpectedConditions.invisibilityOf(succcessMessageNotification1));

//logout from app
wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("button[data-resin-target=\"accountmenu\"]"))).click();
wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("a[data-resin-target=\"logout\"]"))).click();
System.out.println("Loggedout from application");
```

Quiz

- What is the purpose of **Window Interface** and **Navigation Interface**?
- Can you name some important methods of **Window Interface** and **Navigation Interface?**
- What does the **assertions** do and how do they work?
- When will you use **Select** class in **WebDriver** script?
- Which method is used to select the **option with text** in **Select** class?
- How can you wait for **WebElement invisibility**?
- How can you create **Xpath locator** of **parent element** using **child element's** property?



Homework

- Create a **new bookmark WebDriver** script:
  - Navigate to http://app.box.com
  - Verify the **login page title**
  - **Login** into app
  - Verify **Home page title**
  - Click on New and select **Create a Bookmark**
  - Get and **assert** popup text
  - Input **Random** bookmark name
  - Click on "**Create"** button
  - Verify **success notification** and wait for it to disappear from the screen
  - Select newly created **bookmark row.**
  - Click on **Trash button** and then confirm to delete.
  - Verify **deletion notification** and wait for it to disappear from the screen
  - **Logout** from the portal